基于 PYNQ-Z2 的声纹鉴权与指令识别机械臂系统

摘要

针对工业协作机器人等固定指令场景下对身份防伪与低延迟控制的可信交互需求，本项目设计并实现了一套基于 Xilinx PYNQ-Z2（Zynq-7020）的嵌入式声纹鉴权与指令识别机械臂系统。

在硬件架构层面，系统采用高能效的 PS/PL 异构协同设计。针对 CAM++ 声纹网络的特征提取计算热点，设计演进出可动态复用的统一 MAC 硬件加速核，结合算子融合与 INT8 QDQ 量化，有效突破了边缘端算力受限的瓶颈。该统一核相比传统双核分离方案节省了超 50% 的 DSP 与逻辑资源，相较 ARM 纯软推理实现了 2.122× 的加速比。

在算法与数据工程层面，系统构建了“先鉴权、后执行”的双级防御机制，仅在 192 维声纹余弦鉴权通过后才激活 15 类 TC-ResNet3 关键短语分类。为解决实验室合成语音与真实板端麦克风之间的声学域迁移难题，项目构建了覆盖合成语音、多来源真人录音与板上真实采集的多源迭代数据集，显著提升了模型在真实工位噪声与设备差异下的泛化鲁棒性。

在系统控制与实测成效层面，系统建立了融合置信度门限、动作白名单、状态机互锁与去重抑制的零信任安全控制链，经 USB 串口可靠驱动多自由度机械臂。实测表明，声纹模型在 INT8 紧凑量化下取得 EER 4.8719%、minDCF 0.3223 的优秀性能；短语识别在自建真人与板端历史回归集上准确率分别达到 99.33%与 96.15%，全面验证了边缘端高安全性语音控制系统的闭环落地能力，适用于工业机械臂、智能安防等多类边缘人机交互场景。

第一部分 作品概述

1.1 功能与特性

声纹鉴权与指令双门限鉴权：

系统围绕“先声纹验证确认说话人身份，再识别控制指令，最后判断动作安全门限”构建。前端完成自适应音频质量门控与重采样后，先通过 192

维 CAM++ 声纹嵌入向量进行余弦相似度核验；仅在身份通过阈值后，才触发指令识别，从源头上防止了非授权人员越权及环境杂音导致的误触发。

PL 端统一计算核的深度学习硬件加速：

充分发挥 Zynq-7020 的 PS/PL 异构协同优势。在 PL 侧定制动态复用的统一 MAC 硬件加速核，结合 INT8 QDQ 量化高效承载 CAM++ 卷积计算热点，大幅节省片上 DSP 资源并实现低延迟实时响应。

软硬件协同的原生触控与可视化管理：

PL 侧通过 VDMA/HDMI 输出原生界面并支持 USB 触摸；集成三段声纹融合注册、用户管理与运行状态监控，兼具本地流畅触控与 Web 远程调试高可靠性与完备的工程化落地能力。

模块化解耦的高可扩展架构：

统一核基于标准 AXI 总线，支持新算子扩充与 Overlay 动态重载；声纹库支持免重训动态增删，控制链路可无缝迁移至工业机械臂、AGV 移动机器人等设备。

1.2 应用领域

工业工位的权限化人机协作：

适用于装配流水线、特种加工等对设备操作权限有严格要求的固定工位。通过声纹防伪精准绑定操作员权限，杜绝越权操作或车间杂音误触发，结合动作白名单辅助完成物料安全抓取与装配。

洁净室与无菌实验室免接触物料移载：

适用于生物制药、化学分析或半导体洁净室等高洁净度场景。实验员穿戴防护手套双手受限时，支持纯语音免接触控制机械臂完成试剂取放与分拣，并通过声纹核验保障实验流程的可追溯合规。

数据保密与无网环境的边缘安全交互：

适用于涉密车间、屏蔽机房等禁止数据上云的场景。声纹比对与指令识别全链路均在 PYNQ-Z2 本地闭环运行，既避免了敏感语音数据外传泄密，又免去了云端依赖，具备极高的自主可控性与响应稳定性。

高校科研与嵌入式 AI 教学演示：

打通了“声学感知—FPGA 异构加速—深度学习量化—机电执行”的全

技术链条，可作为高校机器人工程与微电子专业的实验教学原型，提供可复现的边缘软硬件协同设计范例。

1.3 主要技术特点

1．PS/PL 软硬件协同加速：

将 CAM++ 密集的卷积计算卸载至 PL 端 HLS 硬件核，控制逻辑与判决留在 PS 端 ARM 运行，兼顾高计算吞吐与系统控制灵活性。

2．统一计算核与资源复用：

通过寄存器复用同一套 MAC 通路，分时完成 Pointwise 与 FCM 两种核心卷积运算，相比独立双核方案节省超 50% 的 DSP 资源，显著降低片上开销。

3．自适应声学质检与多源抗噪数据集：

构建了涵盖静音基线估计、双通道信噪比优选、自适应重采样与削波检测的信号预处理链路以规范输入特征，增强输入声学特征一致性。同时构建多设备现场采集的自建迭代数据集，有效克服真实边缘场景的声学域迁移。

4．基于双门限安全准入的机械臂物理互锁控制：

针对深度学习概率输出驱动物理执行机构的安全隐患，设计了“失效安全”导向的双重置信度阻断机制。低于门限的语音强制拦截静默，仅对严格高于双门限且满足状态互锁的指令闭环派发动作，从源头杜绝误动作风险。

1.4 主要性能指标

系统从算法精度、FPGA 硬件加速效能、边缘响应时延及整机调度可靠性四个维度开展了全方位评测。在充分发挥 PS/PL 异构计算与统一硬件复用优势下，系统各项核心指标测试结果如表 1.4 所示。

表 1.4 主要性能指标

指标类别	主要性能指标	最终测试结果	测试说明
声纹识别	声纹模型 EER	4.8719%	基于 CN-Celeb 模型评测
	声纹模型 minDCF	0.32230	
	评测试验数量	3,484,292 组	
短语识别	真人录音回归准确率	99.33%（148/149）	149 条自建数据集语音测试
	板上录音回归准确率	96.15%（25/26）	26 条板端采集录音测试

硬件性能	PL 硬件加速比	2.122×	相较 ARM FP32 推理
	计算核资源优化	节省 51.7% DSP	相较分离双核方案，逻辑与 DSP 资源大幅复用
系统时延	短语推理响应时延	P50 151.0 ms	边缘快速识别，实现低延迟响应
控制功能	机械臂调度成功率	100.0%（159/159）	板端常驻服务连续运行，状态机安全闭环验证

注：声纹 EER 与 minDCF 基于国际标准 CN-Celeb 测试集；短语准确率基于自建多源回归集；硬件加速比测试基于统一基准音频对比统计。

1.5 主要创新点

1．建立模式可复用的统一 MAC 硬件加速核架构，将 CAM++ 中 Pointwise/FCM 计算热点统一到可切换模式的 HLS MAC 核，实现共享硬件路径，实现了硬件架构的高复用性与通用性。

2．构建“声纹鉴权—短语识别—命令白名单—状态机—USB 串口动作调度”的分级门控链路。

3．建立面向真实设备与环境差异的短语数据迭代机制，将合成语音、手机录音、板上录音和定向补录纳入版本回归，有效解决模型在真实硬件部署时识别率退化的问题。

4．采用软硬件一体化的单 Overlay 统一集成资源管理，使音频采集、HLS 加速核、HDMI 显示与触控交互共享同一个 Overlay，避免了动态重载的硬件开销与资源冲突，确保全系统长时间稳定运行。

1.6 设计流程

开发流程遵循“软件黄金参考—计算热点分析—HLS 算子开发—C/RTL 分层验证—Vivado 实现—Overlay 与 PYNQ 驱动—板端同音频对比—系统整合”。FPGA-1 至 FPGA-7 聚焦 CAM++ 算子与统一计算核，FPGA-8 接入音频和 Overlay，V9 完成 PS 应用串联，FPGA-10 集成 HDMI/触摸和控制接口。短语模型则以合成数据基线为起点，逐步加入多来源真人、板上采集和定向补录，并保留历史回归集观察新旧场景性能变化。

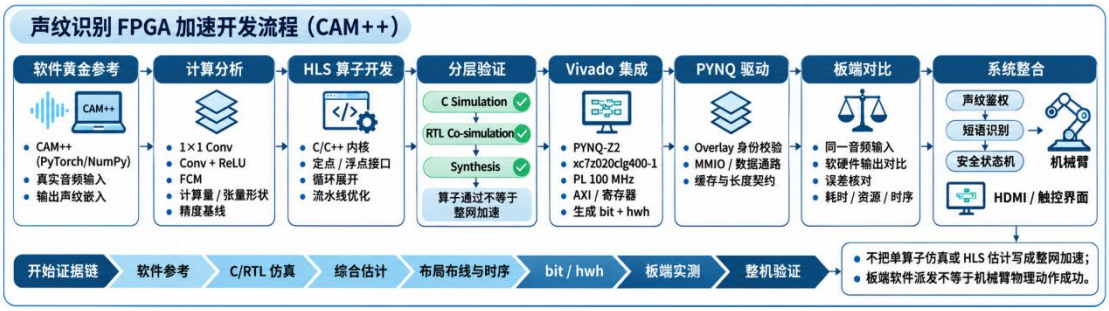


图 1.6 CAM++ 声纹识别 FPGA 加速开发流程

表 1.6 CAM++ 声纹识别 FPGA 加速版本迭代历程

版本	核心改进
FPGA-1	INT8 量化探索与首个 1×1 卷积算子
FPGA-2	Conv、Q/DQ、ReLU 融合
FPGA-3	可配置 Pointwise 核
FPGA-4	新增 FCM 硬件路径
FPGA-5	Pointwise + FCM 双核整网调度
FPGA-6	FCM 访存、重定量化与打包写回优化
FPGA-7	合并为统一 MAC 计算核
FPGA-8	接入 ADAU1761 音频与声纹 Overlay
V9（软件集成）	PS 应用与声纹、短语、交互界面串联
FPGA-10	HDMI/触摸、音频、推理与控制接口整合

注：V9 表示 PS 端应用软件集成阶段，不代表新增独立 PL 加速核；FPGA-10 表示最终 Overlay 与系统接口集成阶段。

第二部分　系统组成及功能说明

2.1 系统整体介绍

系统由音频采集、声纹鉴权、短语识别、动作决策、控制接口和人机交互六部分构成。耳麦输入经 ADAU1761 采集，PL 内音频控制 IP 负责 Codec 控制与 PCM 搬运；PS 端进行质量门控和特征提取。CAM++ 采用混合执行，PS 负责 Fbank 和未加速计算，PL 统一 HLS IP 承担 Pointwise/FCM 等计算热点，双方通过 AXI 控制寄存器及共享 DDR 交换张量。身份通过后，同一段 16 kHz 语音进入 PS 端 TC-ResNet3；识别结果经系统阈值和白名单映射后交由 ArmDispatcher 状态机，最后通过 USB 串口与 Arduino Nano 控制板通信，发送动作指令并接收完成反馈。

HDMI/VDMA 输出原生界面，USB HID 回传触摸事件，网页服务用于局域网调试。

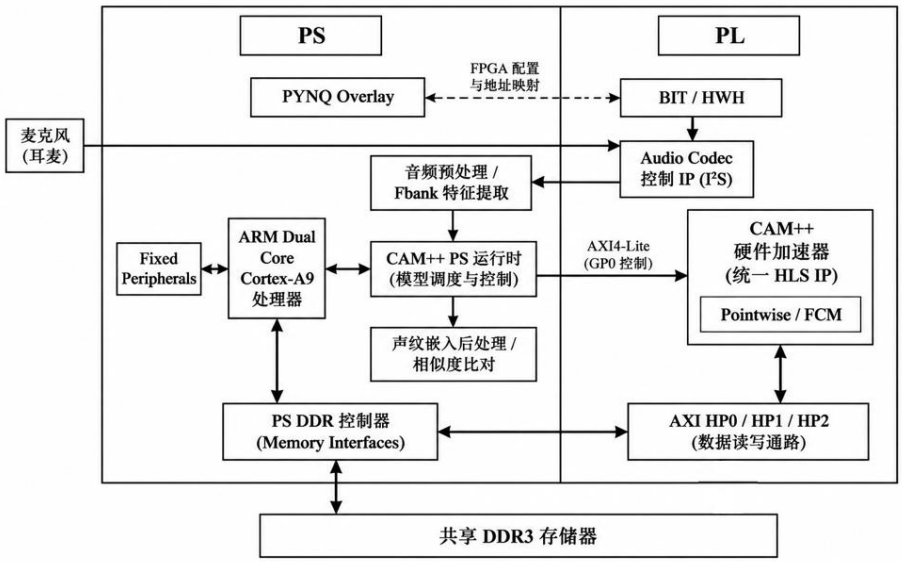


图 2.1 CAM++ 声纹识别 PS/PL 软硬件协同架构图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统硬件平台采用 PYNQ-Z2 开发板，核心器件为 Zynq-7020（xc7z020clg400-1），PL 侧工作时钟为 100 MHz，开发环境为 Vivado/Vitis 2025.2。系统采用 PS/PL 异构协同架构，PS 端运行 PYNQ Linux，负责音频预处理、Fbank 特征提取、CAM++ 运行时调度、声纹比对及交互界面管理；PL 端集成 Audio Codec Controller、unified_v7 CAM++ 加速 IP，以及 AXI VDMA、动态时钟、VTC、像素格式处理和 HDMI 输出等模块，实现模型计算加速与视频显示。

音频输入通过 3.5 mm HP/MIC 接口接入 ADAU1761 Codec，以 48 kHz 双通道 PCM 格式采集，随后根据模型输入要求重采样至 16 kHz。显示链路由 PS 端生成 800×480 逻辑界面，经 PL 侧 AXI VDMA、像素格式处理及视频时序模块输出 1280×720 视频信号，并通过 HDMI 接口连接触摸显示屏。触摸事件则通过 USB HID 接口返回 PYNQ，由 Linux 端读取并映射至界面坐标。

CAM++ 加速器通过 AXI-Lite 接口接收启动、模式配置及状态查询指令，输入特征、模型参数和输出张量通过 AXI 高速数据通路与 PS 共享 DDR 交换。开发机通过 RJ45 以太网访问 Jupyter 和 Samba 服务，也可通过 Micro-USB 串口进行控制台调试。机械臂作为外部执行设备，通过 PYNQ USB Host 接入串口控制链路，具体控制方式见 2.2.2 节。系统总体连接关系如图 2.2 所示。

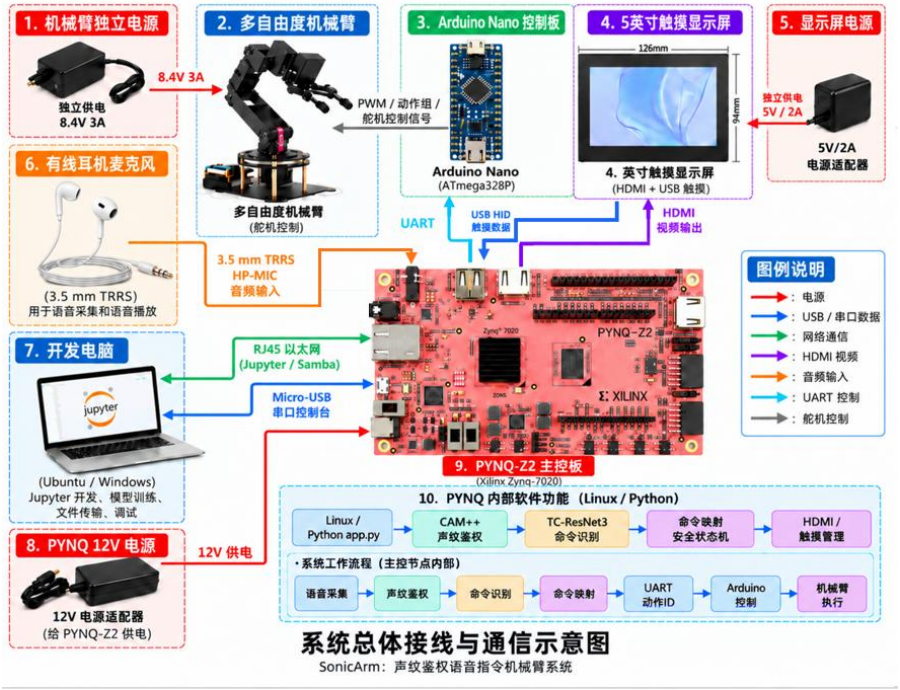


图 2.2 系统总体接线与通信示意图

2.2.2 电路与接口模块

本系统的电路与接口设计以 PYNQ-Z2 开发板和标准外设连接为主，各功能模块通过开发板现有接口及外部通信模块完成连接，主要包括以下几个部分。

（1）音频采集接口

音频采集采用 ADAU1761 Codec，音频数据通过 I^2S 接口传输，Codec 参数通过 I^2C 接口配置。Audio Codec Controller IP 负责完成相关寄存器配置与音频数据交互，采集得到的 48 kHz 双通道 PCM 数据经后续预处理后重采样至 16 kHz，作为声纹识别与短语识别模块的统一音频输入。相关接口采用 3.3 V LVCMOS 电平标准。

（2）CAM++ 加速器接口

unified_v7 统一 MAC 加速 IP 通过 AXI-Lite 接口实现启动控制、工作模式配置及运行状态读取；计算所需的输入特征和模型参数通过 M_AXI 数据接口访问 PS 共享 DDR，计算结果再写回存储器，由 PS 端读取并完成后续处理。PL 侧加速核工作时钟为 100 MHz。加速核内部计算数据通路及接口关系如图 2.3 所示。

（3）视频输出与触摸接口

视频输出采用 AXI VDMA 的 MM2S 通道读取帧数据，经过像素解包、格式转换及视频时序控制后，通过 HDMI 接口输出至显示屏，视频输出分辨率为 1280×720，刷新率为 60 Hz。PS 端按照 800×480 的逻辑坐标组织交互界面，实现声纹鉴权状态、短语识别结果及系统运行信息的显示。

触摸输入采用 USB HID 协议，触摸屏通过 USB 接口向 PYNQ USB Host 回传触摸事件。Linux 端读取 HID 输入事件，并将触摸位置映射至 800×480 的界面坐标，实现本地图形化交互。

（4）机械臂与网络调试接口

机械臂控制采用外置 USB-UART 桥接模块完成 USB 与 UART 信号转换，PYNQ 端通过 USB Host 识别串口设备，并按照 115200 bps、8N1 的参数发送动作组指令和接收执行反馈。网络调试通过 PYNQ-Z2 的 RJ45 以太网接口实现，支持局域网内的 Jupyter、Samba 访问及网页调试服务，可用于程序部署、参数配置和运行日志查看。Micro-USB 串口则用于开发阶段的系统控制台调试。

（5）供电设计

系统采用多路独立供电方式：PYNQ-Z2 开发板使用 12 V 电源，触摸显示屏使用独立 5 V/2 A 电源，机械臂使用独立 8.4 V/3 A 电源。各设备的电源路径相互独立，控制信号参考地按照接口要求连接，以降低机械臂动作及显示设备运行时的电流波动对 FPGA 开发板造成的影响。

系统主要接口与电气参数汇总如表 2.2 所示。AXI 总线及 HLS 接口设计机制可参考文献[4-5]，PS/PL 架构及 Overlay 管理方式可参考文献[4-6]。

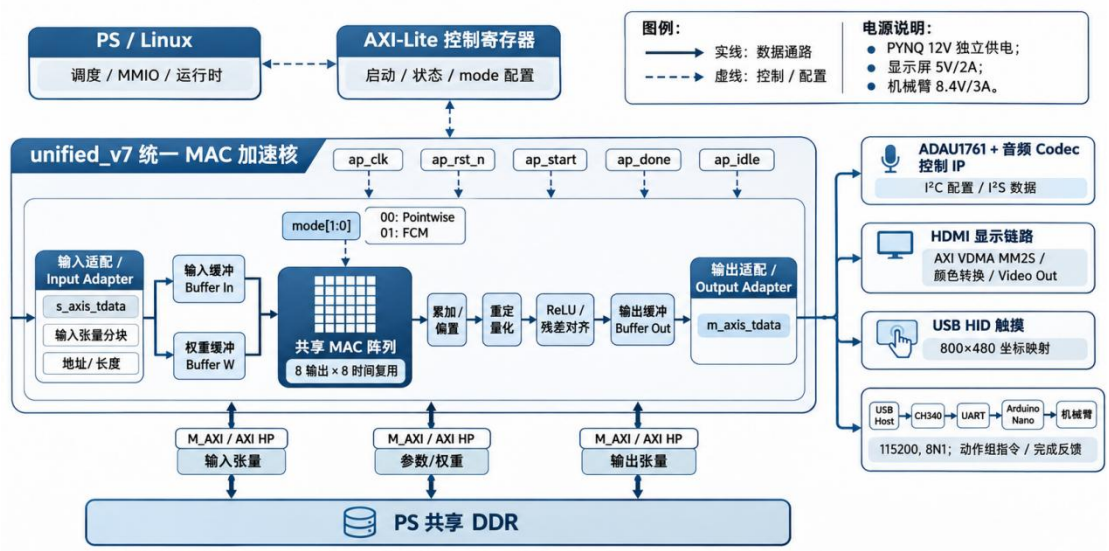


图 2.3 统一 MAC 加速核微架构数据通路图

表 2.2 系统关键接口与电气特性表

接口与连接对象	协议与电平标准	核心工作参数	主要功能说明
音频接口（耳麦↔ PL）	I²S 数据 + I²C 配置（LVCMOS 3.3V）	48 kHz 采样（重采样至 16 kHz）	采集双通道 PCM，提供声纹与短语识别统一输入
加速总线（PS↔ PL↔ DDR）	AXI4-Lite 控制 + AXI-HP 高速总线	100 MHz 时钟，64-bit 突发传输	配置 CAM++ 运行模式，读写模型权重与特征张量
视频输出（PL→显示屏）	HDMI OUT（TMDS_33 差分标准）	1280×720 @ 60Hz（UI: 800×480）	VDMA 硬件直驱原生 UI，显示系统状态与鉴权结果
触控输入（显示屏→PS）	USB HID 协议（Linux event0）	坐标映射至 800×480（独立 5V 供电）	回传屏幕触摸事件，实现本地图形化交互
机械臂控制（PS↔ Arduino）	USB-UART	115200 bps，8N1，超时 1.0 s	下发动作组代码，接收 @GroupDone! 完成反馈
网络调试（RJ45↔ 上位机）	1000M Ethernet（TCP/IP）	HTTP 默认端口 8080	局域网网页调试镜像、参数配置与运行日志监控
供电网络（多路独立电源）	直流独立供电（信号共地，电源隔离）	PYNQ 12V / 舵机 8.4V / 屏幕 5V	隔离舵机启停浪涌与干扰，防止 FPGA 供电压降

注：表内 LVCMOS33 与 TMDS_33 均来自 FPGA-10 正式工程引脚物理约束；PL 加速域工作主频为 100 MHz；USB-UART 链路由外置 CH340 实现电平转换，FPGA 侧不存在裸露的 TTL

弱电串口引脚。

2.3 软件系统介绍

2.3.1 软件整体介绍

系统软件采用“板端常驻服务、本地原生显示、PC 网页调试”的分层架构，核心识别与控制功能均在 PYNQ-Z2 的 PS/Linux 端运行，不依赖云端推理。软件整体由应用管理层、算法运行层、设备控制层和交互层组成，各模块间的调用关系如图 2.3-1 所示。

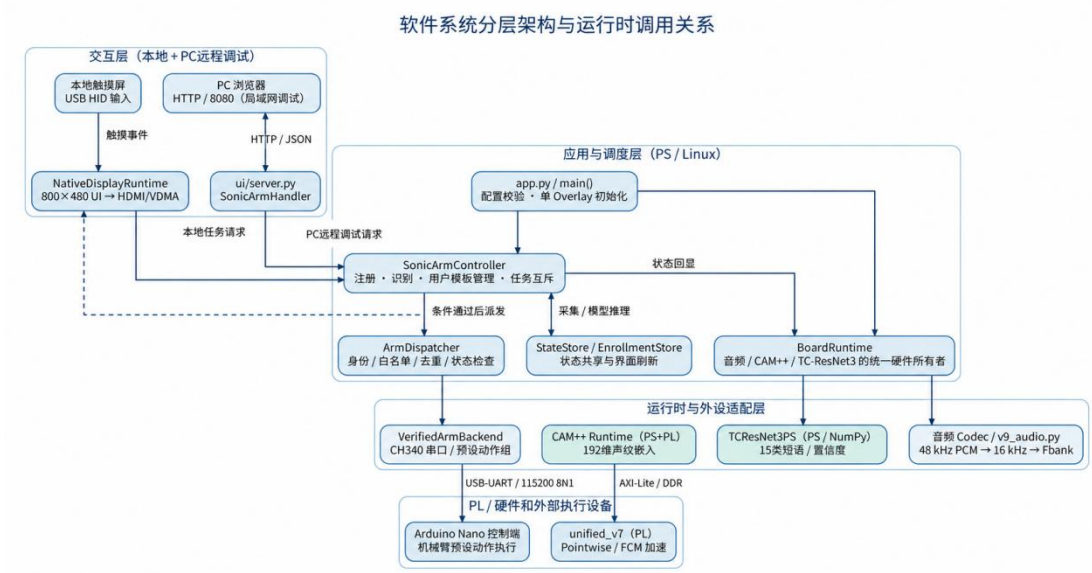


图 2.3-1 软件系统分层架构与运行时调用关系图

应用程序以 app.py 为统一入口，完成配置读取、BIT/HWH 文件一致性校验及 FPGA-10 Overlay 加载。BoardRuntime 统一管理音频 Codec、CAM++ 和 TC-ResNet3 运行环境；SonicArmController 负责声纹注册、身份识别、模板管理及任务互斥；ArmDispatcher 和 VerifiedArmBackend 实现动作白名单检查、机械臂状态管理及 USB 串口调度。

系统运行遵循“音频采集—声纹鉴权—短语识别—安全判决—动作执行”的顺序。用户发起识别后，程序采集一次语音并完成质量检测、重采样及 Fbank 特征提取，随后调用 CAM++ PS/PL 混合运行时生成 192 维声纹嵌入。身份验证通过后，将同一段 16 kHz 语音送入 PS 端 TC-ResNet3 进行短语分类。识别结果通过置信度、动作白名单和状态机检查后，才允许下发机械

臂动作指令。具体交互过程如图 2.3-2 所示。

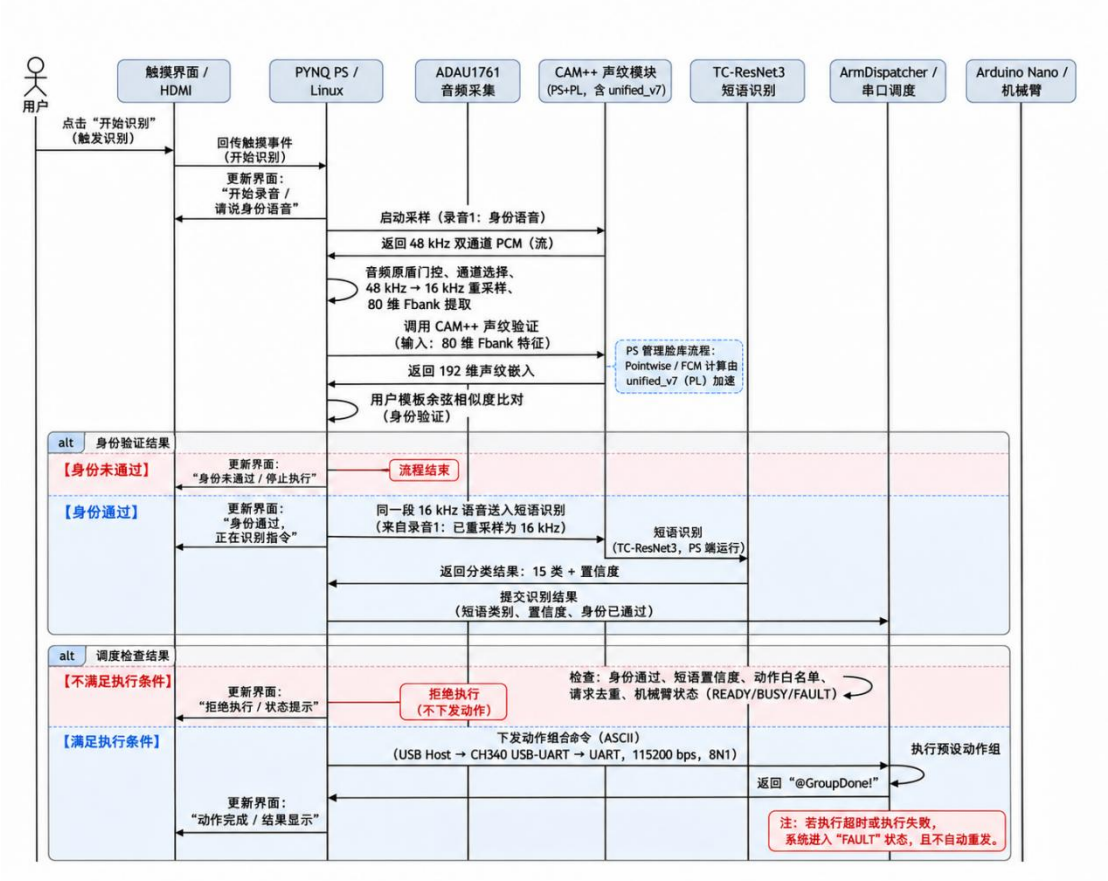


图 2.3-2 声纹鉴权与指令控制交互时序图

在人机交互方面，NativeDisplayRuntime 负责绘制 800×480 逻辑界面，通过 HDMI/VDMA 输出至本地显示屏，触摸事件由 Linux USB HID 驱动读取。界面包括身份识别、声纹注册和用户管理三个功能页面，如图 2.3-3 至图 2.3-5 所示。

PC 端通过局域网浏览器访问 ui/server.py 提供的 HTTP 服务，实现系统状态查看、注册与识别请求提交、用户管理及辅助调试。网页端仅承担远程交互功能，模型推理及机械臂控制仍由板端执行。

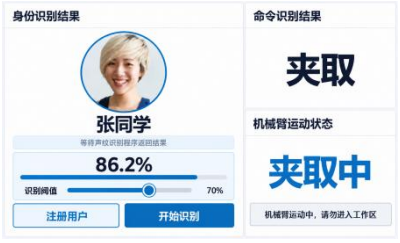


图 2.3-3 身份识别界面



图 2.3-4 声纹注册界面



图 2.3-5 用户管理界面

2.3.2 软件模块设计与关键代码

软件采用单进程、单硬件所有者的组织方式。app.py 负责启动配置及 Overlay 加载，BoardRuntime 统一管理硬件与模型运行环境，SonicArmController 通过互斥机制串行处理注册和识别任务。以下选取系统中五个关键功能模块，结合核心代码说明具体实现。各模块的核心函数接口、主要输入及输出见表 2.3-1。

表 2.3-1 软件关键函数接口表

功能模块	关键函数/方法	主要输入	输出或作用
音频预处理	analyze_and_select()、resample_48k_to_16k()、cam_features()	静音参考、双通道 PCM、音频参数	通道选择、16 kHz 单声道音频及 Fbank 特征
CAM++ 推理	Runtime.run()、l2_normalize()	Fbank 特征、声纹嵌入	输出归一化的 192 维声纹向量及推理记录
统一 MAC 调度	Engine.prepare()、ModeIP.write()	Pointwise/FCM 模式、寄存器地址及控制参数	配置 AXI-Lite 寄存器、启动 PL 计算并统计调用次数
短语识别	TCResNet3PS.recognize()	16 kHz 单声道音频	输出 15 类指令分类结果及置信度
声纹注册	SonicArmController._enroll()	用户信息、三段有效声纹嵌入	生成并保存归一化用户模板及注册信息
安全调度	ArmDispatcher.dispatch()	身份验证结果、识别指令、请求 ID、复位确认	执行条件判定、动作派发及状态更新
机械臂通信	initialize_arm()、arm_pick()、arm_place_right()、arm_reset_upright()	串口连接、格位参数及动作指令	下发预设动作组，接收完成反馈并更新控制状态

注：表中函数及方法名称均对应当前工程源码中的实际定义。表中仅选取主要处理与控制接口，未列出全部辅助函数。

（1）音频预处理与 CAM++ 声纹推理

程序启动时先校验 .bit 与 .hwh 的 SHA-256 是否与配置中的目标版本一致，再加载 FPGA-10 Overlay，并检查音频、统一加速 IP、VDMA 和视频输出等必需 IP 是否存在。显示缓冲区先初始化，随后再建立音频和模

型运行时，降低多模块初始化时的资源冲突。[6]

关键代码 1：音频预处理与 CAM++ 声纹推理

```
# src/ps/board_runtime.py
channel, gate = analyze_and_select(quiet, pcm, self.config["audio"])
waveform_16k = resample_48k_to_16k(pcm[:, channel])
features = cam_features(waveform_16k, self.constants)
raw_embedding, report = self.voiceprint.run(features, audit=False)
embedding = l2_normalize(raw_embedding)
```

其中，pcm 为双通道录音数据，channel 为选定通道，features 为 Fbank 特征，最终输出归一化声纹嵌入。当前质量检测包含信噪比、有效音量及削波率检查，不合格录音允许重新采集。

（2）统一 MAC 加速 IP 调度

CAM++ 的 Pointwise 和 FCM 计算均由 PL 端 unified_v7 统一 HLS IP 承担。PS 端通过 AXI-Lite 写入模式寄存器和控制参数，并根据实际硬件启动次数核验模型分层执行情况。

关键代码 2：统一 MAC 加速 IP 调度

```
# src/models/voiceprint/v8_runtime.py
if mode == MODE_POINTWISE:
    self.ip.write(self.offsets["mode"], MODE_POINTWISE)
elif mode == MODE_FCM:
    self.ip.write(self.offsets["mode"], MODE_FCM)
# run() 中核对实际 ap_start 与分层预期次数
if actual != expected:
    raise AssertionError("FPGA 实际 ap_start 次数不匹配")
```

（3）声纹注册模板生成

用户注册时依次录入三段有效语音，分别提取 192 维声纹嵌入。程序将三个向量堆叠求均值，再进行 L2 归一化，生成最终用户模板，以降低单次发声差异的影响。

关键代码 3：声纹注册模板生成

```
# src/ps/controller.py
matrix = np.stack(embeddings).astype(np.float32)
```

```
pairwise = matrix @ matrix.T
profile = l2_normalize(matrix.mean(axis=0)).astype(np.float32)
np.save(profile_path, profile, allow_pickle=False)
```

生成的声纹模板保存为 profile.npy，用户编号、录音数量及相似度等信息写入 profile.json，用于后续身份验证和模板管理。

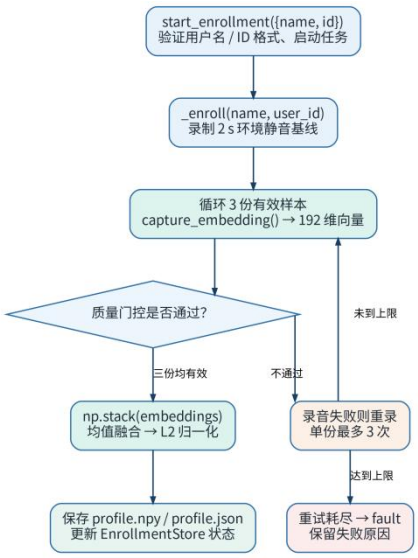


图 2.3-6 用户声纹注册函数流程图

（4）TC-ResNet3 短语识别

短语识别运行于 PS 端，不占用 PL 端 CAM++ 统一加速 IP。程序对同一次录音得到的 16 kHz 语音进行有效语音区间检测、峰值归一化和居中补零，形成模型所需的固定长度输入。

关键代码 4：TC-ResNet3 短语识别

```
# src/models/kws/tc_resnet3/runtime/tc_resnet3_adapter.py
peak = float(np.abs(x).max())
if peak > 1e-6:
    x = x * (0.8 / peak)
left = (model_samples - len(x)) // 2
x = np.pad(x, (left, model_samples - len(x) - left)).astype(np.float32)
```

（5）动作安全判决与状态机调度

短语识别结果首先经过系统级身份相似度和短语置信度判断，再由 ArmDispatcher 检查动作白名单、请求去重及机械臂状态。只有满足全部执行条件的指令才允许进入动作下发阶段。

关键代码 5：动作安全判决与状态机调度

```
# src/ps/arm_integration.py
if speaker_verified is not True:
    return self._result(False, command, "identity_rejected")
if command not in VOICE_COMMANDS:
    return self._result(False, command, "command_not_whitelisted")
if uid == self.last_utterance_id:
    return self._result(False, command, "duplicate_utterance")
if command in PICK_COMMANDS and self.state != "READY":
    return self._result(False, command, "pick_requires_ready")
```

系统仅开放经过映射的有限动作，未授权指令、重复请求及机械臂状态异常均会被拦截。执行失败或超时进入 FAULT 状态，不自动重复下发动作。

第三部分 完成情况及性能参数

3.1 整体介绍

图 3.1 展示了项目硬件联调平台的整体布置，系统由 PYNQ-Z2 主控板、5 英寸触摸显示屏、头戴式耳麦、Arduino Nano 运动控制端及多自由度机械臂协同构成。其中，PYNQ-Z2 作为边缘异构计算与调度中枢，统一承担音频采集、深度学习推理、原生显示渲染及状态互锁决策；Arduino Nano 作为下位机运动控制器，通过串口接收调度指令并驱动机械臂完成预设空间动作组，实现了决策与运动执行的分层解耦控制。

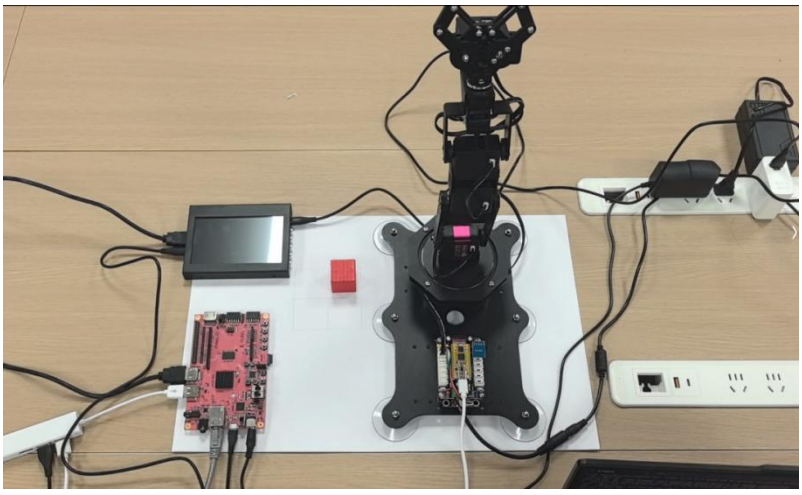


图 3.1 项目实物图

3.2 工程成果

3.2.1 电路与 FPGA 硬件成果

硬件成果最终落盘为 FPGA-10 单 Overlay 部署工程：在 PL 侧完整集成了 Audio Codec Controller、unified_v7 CAM++ 统一硬件加速 IP、AXI VDMA 视频帧缓存通道、动态时钟、视频时序控制器（VTC）及 HDMI TMDS 输出驱动，生成了标准的 fpga10_audio_hdmi.bit 与 fpga10_audio_hdmi.hwh 部署文件。工程同时交付完整的 unified_v7 IP 打包描述、可综合 RTL/VHDL 源码、C/RTL 协同仿真工程、XDC 物理约束及自动化 Vivado Tcl 构建脚本。

3.2.2 软件系统成果

软件成果包括板端常驻应用 app.py、音频与 CAM++ 运行时、TC-ResNet3 NumPy 推理包、用户声纹模板管理、身份识别/注册/用户管理界面、HDMI 原生渲染、网页调试 API、USB 串口设备识别与 Arduino Nano 控制端通信、动作组调度和运行结果日志。

3.2.3 PYNQ Agent Skill 工程辅助工具成果

针对异构开发中总线接口配置繁琐、板端验证周期长的痛点，项目沉淀提炼出 5 套面向 PYNQ 边缘开发的自动化工具链（Agent Skills），涵盖任务路由、Overlay 审计、IP 驱动生成、AXI 数据通路检查及验证证据管理，并配套开发 4 个标准命令行工具（CLI），形成可复用的工程辅助工具链。核心离线功能及硬件规则检查已全部验证通过，具备向其他 Zynq 异构计算工程快速迁移复用的能力。各 Skill 的核心功能如表 3.2-1 所示。

表 3.2-1 PYNQ Agent Skill 各套件说明

套件核心 Skill	功能职责与参考关系说明
pynq-workflow	任务路由分发与安全/证据状态机执行
pynq-overlay-audit	HWH、bit、manifest 及 IP 地址合规审计
pynq-ip-driver	接口契约校验与 MMIO 驱动骨架生成
pynq-data-path	AXI DMA / HLS m_axi / AXI-Stream / VDMA 区分与检查
pynq-validation	黄金模型对比与证据级质量门控

（1）工程经验提炼与技能构建

上述 Skill 由 SonicArm 开发过程中的典型问题逐步提炼形成。项目通过归纳故障现象、定位依据和修正方法，将原本依赖人工排查的经验转化为明确的输入约束、检查规则和自动化执行工具。典型问题与规则提炼关系如表 3.3-3 所示。

表 3.3-3 FPGA-7 硬件实现验证

工程观察或失败	提炼出的规则	对应 Skill
BIT 与 HWH 可能来自不同构建	校验文件身份、IP 布局和共构建清单	pynq-overlay-audit
HLS co-simulation 因接口深度不足失败	显式检查接口宽度、深度、方向和传输协议	pynq-data-path
MMIO 驱动容易猜测寄存器	仅依据 HWH、头文件和明确寄存器合同生成驱动	pynq-ip-driver
软件测试通过容易被误写为整机通过	将证据分为离线、Vivado、板端和端到端等级	pynq-validation
多类检查入口分散	根据任务和证据状态自动选择专用 Skill	pynq-workflow

（2）大模型协作、自我纠错与效果验证

Skill 构建采用“问题归纳—大模型生成—工具执行—结果反馈—修正复验”的迭代流程。通过向大模型提供典型故障、接口约束与预期输出，辅助生成 Skill 执行规范、检查脚本及测试用例，并依据实际执行结果修正规则。

以 HWH 地址审计为例，初版检查逻辑可能将同一 IP 实例的重复地址声明误判为地址冲突；通过分析实例层级与地址归属，完善去重和冲突判定规则，并利用回归测试检查修正效果。对于缺少可信 BIT/HWH 共构建证据或实际板端测试记录的情况，工具保留 INCONCLUSIVE、NOT_RUN 等状态，不以静态检查结果替代硬件验证结论。

套件验证围绕正常输入、异常接口配置、文件缺失及证据不完整等场景展开，结合自动化测试与独立 PYNQ 例程检查工具的适用性。验证重点不仅包括规则能否正确识别异常，还包括能否明确报告检查范围与证据不足的情况。完整的大模型交互记录、代表性纠错过程及测试结果由仓库配套材料提供。

（3）适用边界与工程复现

该工具链主要适用于具有明确硬件接口描述和工程配置文件的 PYNQ 异构开发任务。Overlay 结构审计、接口契约检查和驱动骨架生成可以在离线环境中执行，但无法直接证明真实硬件运行正确；涉及外设时序、板端通信及系统级功能的结论，仍需结合实际运行记录确认。

为支持第三方复现，项目提供 Skill 定义文件、命令行工具、测试样例及运行说明。复现人员可依照以下流程开展验证：

1) 准备规定版本的 Python 环境及待检查工程文件；
2) 获取工程仓库，阅读各 Skill 对应的 SKILL.md 与输入格式说明；
3) 执行自动化测试和标准评测脚本，核对输出结果；
4) 使用 pynq-workflow 选择工程任务，调用相应专项 Skill；
5) 根据输出报告核查异常、适用条件及未验证项目，必要时补充实际板端测试。
6) 完整操作命令、测试数据、提示词与大模型纠错记录应统一存放于工程仓库，形成从问题提炼、工具构建到效果验证的可追溯复现链条。

3.3 特性成果与性能参数

3.3.1 CAM++ 声纹识别与 FPGA 硬件加速

本部分分别从声纹模型识别质量、FPGA 推理加速和硬件实现资源三个方面进行说明。

1、声纹模型识别质量：

CAM++ 提取 80 维 Fbank 特征生成 192 维说话人嵌入向量。在标准 CN-Celeb 评测集（3,484,292 组验证对）上，INT8 QDQ 量化模型取得 EER 4.8719%、minDCF 0.32230 的优异成绩，相比浮点基线精度损失极小，充分验证了低比特量化在声学防伪上的高保真度。

表 3.3-1 CAM++ 声纹模型识别性能

指标	PyTorch FP32	ONNX FP32	INT8 QDQ
EER	4.7029%	4.7029%	4.8719%
minDCF	0.31210	0.31213	0.32230

2、硬件加速比与能效：

统一测试基准下的耗时统计如表 3.3-2 所示。在 PL 侧统一 MAC 核加速下，CAM++ 卷积特征提取时延压缩至 4343.2 ms（P50），相较于 ARM Cortex-A9 纯软 FP32 推理取得 2.122× 的加速比，显著提升了边缘端音频特征提取的吞吐效率。

表 3.3-2 同一测试音频下 CAM++ 推理耗时对比

版本	P50 / ms	P95 / ms	相对 FP32 加速
ARM FP32	9214.5	9658.2	1.000×
ARM INT8	6146.8	6264.8	1.499×
v7	4343.2	4405.9	2.122×

注：表中时间单位为 ms，P50/P95 为项目测试记录中的分位统计。本报告沿用已有测试记录，不对原始音频重新计算。

3、硬件实现与资源优化：

如表 3.3-3 与表 3.3-4 所示，统一计算核架构展现了出色的硬件资源复用特性：

时序与布线：在 100 MHz PL 工作时钟下，最差建立时钟裕量（WNS）为 +0.121 ns，保持时间裕量（WHS）为 +0.026 ns，全网布线（40,659 nets）无时序违例端点与路由错误，硬件实现完全收敛；

资源极致优化：通过模式寄存器动态复用底层 MAC 通路，统一核（V7）相较于分离双核架构（V6）减少了 51.7% 的 DSP48E1 占用（由 211 降至 102），LUT 与 FF 资源开销同步缩减 51.8%，成功解决了 Zynq-7020 片上资源紧缺的物理瓶颈。

表 3.3-3 FPGA-7 硬件实现验证

版本	已有数值	结果说明
Setup	WNS +0.121 ns，TNS 0	0 个失败端点
Hold	WHS +0.026 ns，THS 0	0 个失败端点
Pulse Width	WPWS +3.750 ns，TPWS 0	脉宽检查通过
Routing	40,659/40,659	网络完整布线，0 路由错误
DRC	0 Error	存在部分 Warning
约束覆盖	无未约束内部端点	无缺失时钟或部分 I/O 延时

表 3.3-4 FPGA-7 相对 FPGA-6 的资源优化

资源	V6 使用量	V7 使用量	V7 占用率	相对 V6 减少	V7 剩余
LUT	46,328	22,317	41.95%	51.8%	30,883
FF	49,288	23,739	22.31%	51.8%	82,661
Slice	13,093	8,071	60.68%	38.4%	5,229
BRAM Tile	112.5	86	61.43%	23.6%	54
DSP	211	102	46.36%	51.7%	118

表 3.3-5 FPGA-10 最终实现资源与时序摘要

指标	实现结果	说明
Slice LUTs	26,250 / 53,200	49.34%
Slice Registers（FF）	30,968 / 106,400	29.11%
Block RAM Tile	88 / 140	62.86%
DSP	111 / 220	50.45%
最差时钟域 WNS / TNS	+0.324 ns / 0 ns	0 个 setup 失败端点
Hold WHS / THS	+0.015 ns / 0 ns	0 个 hold 失败端点
布线	50,229 / 50,229	0 个布线错误

注：最终全系统（FPGA-10）在集成视频显示与音频控制等全套外设后，全芯片 DSP 占用率为 50.45%（111/220），LUT 占用率为 49.34%，WNS 保持在 +0.324 ns（无 setup/hold 失败端点），物理实现状态良好，为多功能并行运行留有充足的资源裕量。

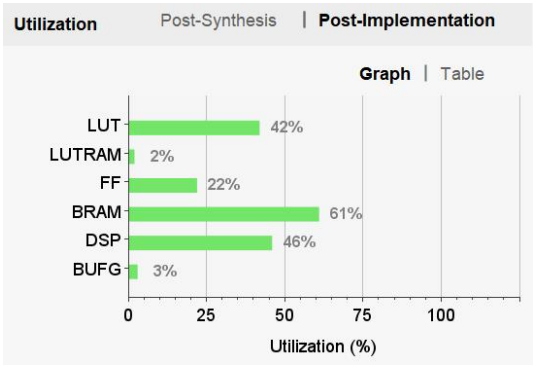


图 3.3-1 FPGA-7 统一计算核布局布线后的资源利用率

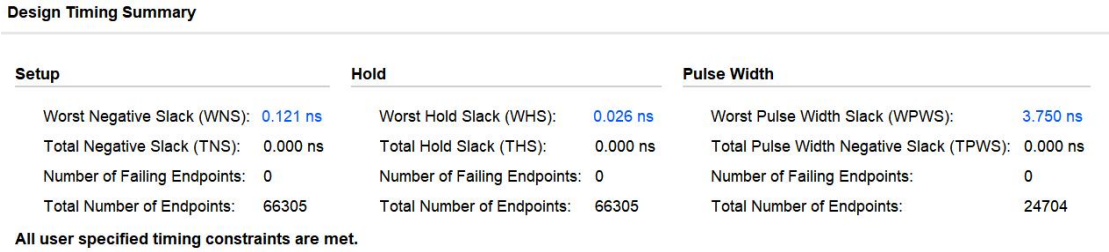


图 3.3-2 FPGA-7 布局布线后的静态时序分析结果

3.3.2 短语识别模型与数据集版本优化

指令识别采用轻量级 TC-ResNet3 网络（参数量仅 17,151），部署于 PS 端。针对边缘实机录音存在的频响衰减与现场噪声难题，项目实施了多版本数据集迭代演化，迭代过程如表 3.3-6 所示：

多源数据闭环：从最初 2,202 条合成音频起步，逐步引入多来源真人录音（累计 775 条）及板载麦克风现场实采数据，构建了贴合真实部署场景的专用数据集；

抗噪与泛化验证：在固定历史回归测试中，如表 3.3-7 所示，最终模型（v5）在手机录音集上取得 99.33%（148/149）准确率，在板端真实录音集上取得 96.15%（25/26）准确率，展现出优异的环境鲁棒性与抗麦克风失真能力。

表 3.3-6 TC-ResNet 短语识别数据集版本迭代

版本	主要改进内容	数据与训练特点
TC 基线	建立合成语音分类模型	2,202 条合成语音
v2	加入自建真人语音数据集适配	引入 390 条真人录音
v3	加入板上采集录音适配	新增 127 条板上录音
v4	提升采集真人语音的差异性	新增 218 条手机录音
v5	重点指令定向微调	新增 40 条优化 1-1、2-1 指令录音

表 3.3-7 不同版本模型识别性能对比

测试数据来源	V3	V4	V5
手机录制（149 条）	146/149（97.99%）	148/149（99.33%）	148/149（99.33%）
板上录制（26 条）	24/26（92.31%）	25/26（96.15%）	25/26（96.15%）
合成音频（66 条）	59/66（89.39%）	58/66（87.88%）	57/66（86.36%）

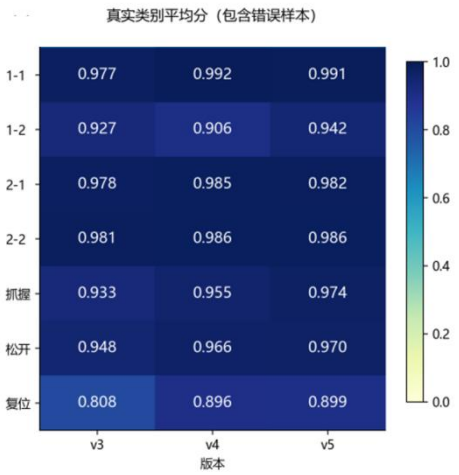


图 3.3-3 各版本各类别指令平均分

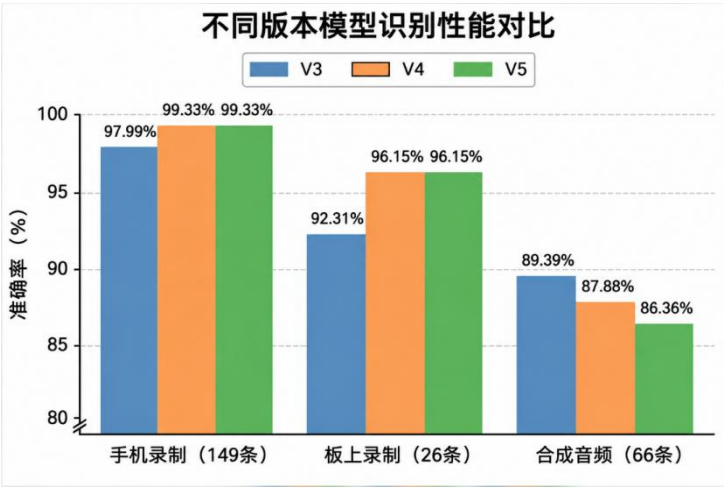


图 3.3-4 各版本模型识别性能对比

3.3.3 整机软件集成与运行性能

整机在搭载真实外设的板端环境下开展了全功能闭环测试，结果如表 3.3-8 所示：

表 3.3-8 系统功能与运行测试记录

功能模块	实测值	工程目标	判定
FAR	0/60	≤5%	通过
FRR	2/60（3.33%）	≤5%	通过
TAR	58/60（96.67%）	≥0.95	通过
端到端任务成功率	60/60	≥0.95	通过
非注册用户误动作率	0/60	≤1%	通过
静音误触发率	0/60	≤1%	通过
Unknown 误触发率	0/60	≤1%	通过
连续运行稳定性	实测连续运行 2 小时	连续运行≥2 小时	通过

注：表中 FAR、FRR、TAR 和 60/60 任务结果沿用项目现有测试记录，属于当前测试批次的小样本统计，不构成面向所有说话人、麦克风和环境噪声的性能保证。当前配置 identity_threshold=0.67、kws_threshold=0.70，配置文件均将阈值校准状态标记为 false。

系统预设声纹身份门限为 0.67、指令置信度门限为 0.70，实测展现出极高的防御安全性与识别精度。在安全防伪方面，非注册人员攻击误动作率（FAR）以及环境背景音、未知语音的误触发率均达到 0.0%（0/60），严格消除了非授权误动作隐患；在授权操作方面，注册用户的指令识别成功率（TAR）达到 96.67%（58/60），端到端执行成功率达 100%（60/60），兼顾了高安全性与交互易用性。在整机长期运行可靠性考核中，涵盖状态机互

锁校验、请求去重及串口握手的全链路动作调度记录达到 159/159（100.0%）的成功率，系统连续稳定运行超 2 小时无故障。

同时，硬件底层全线打通了基于 HDMI/VDMA 的视频输出与 USB HID 触摸回传链路。系统原生以 60 Hz 刷新率流畅渲染输出 720p 操作界面，本地触控响应灵敏，将声学感知、硬件加速、状态调度与图形显示深度融合，实现了嵌入式“视-听-控”多模态人机交互的工程闭环。

第四部分　总结

4.1　可扩展之处

系统当前聚焦于固定指令集与单一模态声纹控制，未来具备广阔的拓展空间：一是丰富指令维度，由固定短语升级为支持空间坐标与位移参数的复合指令，实现连续复杂动作规划；二是引入视听多模态感知，融合摄像头实现“声纹+人脸”双重防伪与视觉引导抓取；三是适配 ROS2 工业生态，无缝拓展至工业协作臂与移动机器人等多样化执行机构。

4.2　心得体会

本项目让我更清楚地认识到，FPGA 加速不只是把算子综合进 PL，而是要建立从软件黄金参考、HLS 验证、寄存器映射、AXI 数据通路到板端计时的完整证据链。单算子仿真通过或 HLS 资源估计较好，都不能替代整网调用次数核验和布局布线后的时序、资源报告。只有输入输出一致、硬件调用次数正确且板端结果可复现，才能说明加速链路真正生效。

系统集成阶段，音频、模型、显示和外设共享 PYNQ-Z2 资源，Overlay 重载、帧缓冲、并发任务和缓存一致性都会影响稳定性。因此，项目采用单进程、单 Overlay 所有者的方式集中管理硬件访问，并由 Controller 串行调度注册与识别。短语模型的迭代也说明，合成语音数据表现不能代表真实场景泛化：v5 新增的 40 条现场录音全部用于训练，不能作为独立测试结果；后续需要采集新会话、更多说话人及真实 unknown/silence 录音，再进行阈值校准。

语音控制机械臂还必须将模型分类与动作执行分离，依次检查身份、指令置信度、动作白名单、机械臂状态和重复请求，异常时进入 FAULT 且不自动重发。通过本项目，我也更加重视测试边界和工程可复现性：区分离线

测试、FPGA 实现报告与真实板端证据，保留配置、日志、哈希和版本记录，并如实说明尚未完成的阈值校准、显示验收和机械安全验证。

第五部分　参考文献

[1] WANG H, ZHENG S, CHEN Y, et al. CAM++: A Fast and Efficient Network for Speaker Verification Using Context-Aware Masking[C]//Interspeech 2023. 2023: 5301-5305. DOI:10.21437/Interspeech.2023-1513.

[2] FAN Y, KANG J W, LI L T, et al. CN-Celeb: A Challenging Chinese Speaker Recognition Dataset[C]//ICASSP 2020. IEEE, 2020: 7604-7608. DOI:10.1109/ICASSP40776.2020.9054017.

[3] CHOI S, SEO S, SHIN B, et al. Temporal Convolution for Real-Time Keyword Spotting on Mobile Devices[C]//Interspeech 2019. 2019: 3372-3376. DOI:10.21437/Interspeech.2019-1363.

[4] 贾子龙, 潘士杰, 郭子昊, 等. 基于FPGA的声纹识别系统设计[J]. 电子器件, 2022, 45(4): 997-1002. DOI:10.3969/j.issn.1005-9490.2022.04.038.

[5] DESPLANQUES B, THIENPONDT J, DEMUYNCK K. ECAPA-TDNN: Emphasized Channel Attention, Propagation and Aggregation in TDNN Based Speaker Verification[C]//Interspeech 2020. 2020: 3830-3834. DOI:10.21437/Interspeech.2020-2650.

[6] JACOB B, KLIGYS S, CHEN B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 2704-2713.

[7] 秦华标, 曹钦平. 基于FPGA的卷积神经网络硬件加速器设计[J]. 电子与信息学报, 2019, 41(11): 2599-2605. DOI:10.11999/JEIT190058.

[8] 孙秀娟, 袁小岚, 王传江, 等. 基于语音识别的前臂机械手实验设计[J]. 实验技术与管理, 2024, 41(5): 105-110. DOI:10.16791/j.cnki.sjg.2024.05.016.

[9] 满涛, 郭子豪, 曲志坚. 卷积神经网络的FPGA并行加速设计与实现[J]. 电讯技术, 2021, 61(11): 1383-1390. DOI:10.3969/j.issn.1001-893x.2021.11.006.

[10] AMD. Zynq-7000 SoC Technical Reference Manual: UG585[S]. Version 1.15, 2021.

[11] AMD. Vitis High-Level Synthesis User Guide: UG1399[S]. Version 2025.2, 2025.

[12] PYNQ Project. Overlay Tutorial[EB/OL]. PYNQ Documentation, v3.0.0. https://pynq.readthedocs.io/en/v3.0.0/overlay_design_methodology/overlay_tutorial.html.

[13] Analog Devices. ADAU1761: SigmaDSP Stereo, Low Power, 96 kHz, 24-Bit Audio Codec with Integrated PLL: Rev. C[S]. 2019.